

APRENDIZAJE DE MÁQUINAS (ACIF104) INFORME FASE 3 — SUMATIVA 2

Nombre integrante/s:

Javiera Chávez
Joaquín Olivares
Claudio Yáñez

Curso: Aprendizaje de Máquina

Fecha: 08-09-2026

Índice

1. Problemática	4
1.1 Requisitos iniciales	4
1.2 Marco de alcance	5
2. Metodología.....	6
2.1 Objetivos del proyecto.....	6
2.2 Enfoque metodológico.....	6
2.3 Fases del desarrollo	7
2.4 Plan de trabajo	8
3. Desarrollo del proyecto.....	9
3.1 Análisis exploratorio de datos.....	9
3.1.1 Descripción de las variables	9
3.1.2 Completitud y correctitud.....	9
3.1.3 Estadística descriptiva.....	9
3.1.4 Valores atípicos.....	10
3.1.5 Correlación entre variables	11
3.1.6 Balance de clases	12
3.2 Selección de técnicas candidatas.....	13
3.3 Comparación de técnicas	13
3.4 Requisitos del proyecto	15
3.4.1 Requisitos funcionales	15
3.4.2 Requisitos no funcionales	16
3.5 Selección de la arquitectura del modelo	16
3.5.1 Arquitecturas de Deep Learning y análisis de convergencia	16
3.5.2 Refinamiento del modelo seleccionado	17
3.6 Elaboración del modelo: partición y balanceo.....	18
3.7 Desarrollo frontend y backend	18
3.7.1 Capturas del sistema en funcionamiento	19
4. Resultados.....	21
4.1 Matriz de confusión	21
4.2 Análisis interpretativo con SHAP	21
4.3 Cumplimiento de objetivos y requisitos	23
5. Conclusiones, limitaciones y trabajos futuros	24
5.1 Limitaciones	24

5.2 Mejoras propuestas	24
5.3 Resumen de hallazgos y aprendizajes.....	24
5.4 Aplicabilidad del sistema	25
6. Código fuente en GitHub	26
7. Bibliografía	27

1. Problemática

Con la Industria 4.0, las plantas industriales se llenaron de sensores. Hoy una máquina puede registrar de forma continua su temperatura de aire y de proceso, su velocidad de rotación, el torque y el desgaste de la herramienta, y todos esos datos quedan almacenados (Hamasha et al., 2025). El problema es que tener los datos no significa aprovecharlos. Muchas empresas siguen trabajando con mantenimiento correctivo, es decir, reparar la máquina una vez que se rompió, o con mantenimiento preventivo, revisándola cada cierto tiempo aunque esté funcionando bien. Las dos formas resultan costosas: la primera provoca paradas inesperadas y reparaciones de emergencia, y la segunda lleva a reemplazar componentes que todavía tenían vida útil (Benhanifia et al., 2025). Meitz et al. (2025) señalan que el mantenimiento puede representar entre un 15 % y un 70 % del costo total de producción según el rubro, lo que muestra cuánto está en juego.

El mantenimiento predictivo ofrece una alternativa: usar los datos de operación de cada equipo para estimar cuándo es probable que falle y programar la intervención justo antes de que ocurra. Es una de las aplicaciones más frecuentes del aprendizaje automático en la industria (Benhanifia et al., 2025; Guidotti et al., 2025). La dificultad está en encontrar, dentro de todo ese volumen de datos, las señales que anticipan una falla, algo que no se puede hacer de forma manual (Guidotti et al., 2025).

Al revisar la literatura reciente se nota una diferencia de enfoque entre los autores. Las revisiones sistemáticas de Guidotti et al. (2025) y Meitz et al. (2025) plantean que el problema pendiente ya no son los algoritmos, que funcionan bien, sino otras dos cosas: existen pocos conjuntos de datos industriales públicos para comparar y reproducir resultados, y los modelos suelen comportarse como cajas negras que el personal de mantenimiento no logra interpretar ni en las que confía. Los trabajos de tipo aplicado apuntan en otra dirección. Aminzadeh et al. (2025), que implementaron un sistema real en compresores industriales, concluyen que un modelo preciso aporta poco si no está integrado en una herramienta que el personal pueda usar en su trabajo diario. Hamasha et al. (2025) proponen un marco que combina inteligencia artificial, IoT y edge computing, pero se mantiene en un plano conceptual. Si se juntan las dos miradas, queda a la vista un vacío: hay pocos trabajos que entreguen al mismo tiempo un sistema completo y funcional, que explique sus predicciones y que esté construido sobre datos públicos reproducibles. Ese es el vacío que este proyecto busca abordar.

El caso que da origen al proyecto es el de una planta manufacturera que opera equipos rotativos y de mecanizado. En ese contexto, el equipo de mantenimiento, y en particular el supervisor, decide qué máquinas inspeccionar apoyándose sobre todo en su experiencia y en la pauta de calendario. Ese conocimiento, saber que cierta combinación de lecturas suele terminar en falla, no queda registrado en ningún sistema y se pierde cuando la persona cambia de turno o de puesto. Perfectime apunta a ese problema. Funciona como una herramienta de apoyo a la decisión y de gestión del conocimiento: recibe las lecturas de sensores de una máquina, entrega la probabilidad de falla y el nivel de riesgo asociado, y muestra mediante SHAP qué variables influyeron más en esa estimación. De esta forma el supervisor puede ordenar las inspecciones según un criterio cuantificable y no solo según su intuición.

1.1 Requisitos iniciales

De los antecedentes revisados se desprenden los requisitos con los que partió el proyecto:

Antecedente	Requisito inicial
Revisar manualmente grandes volúmenes de telemetría para detectar las señales previas a una falla no es viable (Guidotti et al., 2025)	El sistema estima automáticamente la probabilidad de falla de una máquina a partir de sus lecturas de sensores
Una solución de mantenimiento predictivo aporta valor solo si se integra en una plataforma que el personal pueda operar (Aminzadeh et al., 2025)	El sistema entrega una interfaz web para el supervisor (predicción individual y por lotes) apoyada en un servicio/API
La falta de interpretabilidad frena la adopción de estos modelos (Guidotti et al., 2025; Meitz et al., 2025)	Cada predicción se acompaña del aporte de cada variable, calculado con SHAP (requisito de explicabilidad)
No detectar una falla tiene un costo alto (Meitz et al., 2025)	La evaluación prioriza la detección de la clase “con falla” (Recall y F1) por sobre la exactitud general
El desbalance de clases es propio de este tipo de problema (He & García, 2009)	El entrenamiento incorpora y compara distintas técnicas de balanceo
Hay escasez de datasets industriales públicos, lo que dificulta reproducir resultados (Guidotti et al., 2025)	El proyecto se construye sobre un conjunto de datos público y documentado (AI4I 2020; Matzka, 2020), en un entorno reproducible
Auditar las decisiones de mantenimiento exige trazabilidad	El sistema registra las predicciones realizadas y permite seguir el estado del modelo (monitoreabilidad y confiabilidad)

Tabla 1. Requisitos iniciales derivados de los antecedentes.

1.2 Marco de alcance

El sistema se acota a lo siguiente:

- Clasificación binaria del riesgo de falla (falla o no falla) a partir de siete variables de entrada: el tipo de producto y cinco lecturas de sensores.
- Explicación de cada predicción con valores SHAP.
- Predicción individual y por lotes mediante carga de archivo CSV.
- Interfaz web para el supervisor y servicio REST que sirve el modelo.
- Registro histórico de predicciones y monitoreo básico del sistema.
- Reentrenamiento del modelo con nuevos registros etiquetados.

Quedan fuera del alcance:

- Identificar el tipo específico de falla (TWF, HDF, PWF, OSF, RNF). Esas variables solo se conocen después de que la falla ocurrió, así que usarlas sería fuga de datos.
- Estimar la vida útil restante del equipo (RUL).
- Ingerir telemetría en tiempo real desde dispositivos IoT. El sistema trabaja con lecturas puntuales que se ingresan o se cargan.
- Despliegue en varias plantas y optimización de la programación de mantenimiento.

2. Metodología

2.1 Objetivos del proyecto

Objetivo general. Desarrollar y validar un sistema de mantenimiento predictivo que, a partir de los datos de sensores de un equipo industrial, estime su probabilidad de falla de forma explicable, y ponerlo a disposición en una plataforma web para el supervisor de mantenimiento.

Objetivos específicos.

1. Analizar el dataset AI4I 2020 y evaluar su calidad antes del modelado.
2. Comparar técnicas de aprendizaje automático y arquitecturas de aprendizaje profundo, y seleccionar el modelo con mejor desempeño según métricas alineadas al problema.
3. Evaluar cómo afectan distintas técnicas de balanceo de clases al rendimiento del modelo.
4. Optimizar y validar el modelo seleccionado sobre datos no vistos.
5. Interpretar las predicciones del modelo con SHAP.
6. Integrar el modelo con un backend y un frontend que permitan predicción individual y por lotes, historial, monitoreo y reentrenamiento.

2.2 Enfoque metodológico

El proyecto se desarrolló siguiendo CRISP-DM (Cross Industry Standard Process for Data Mining), una metodología muy usada en proyectos de análisis de datos y aprendizaje automático (Chapman et al., 2000). CRISP-DM divide el trabajo en seis etapas: comprensión del negocio, comprensión de los datos, preparación de los datos, modelado, evaluación y despliegue. Se eligió porque se ajusta bien a un proyecto de este tipo: parte del problema real antes de tocar los datos, separa con claridad la preparación de datos del modelado, y permite volver a una etapa anterior cuando los resultados lo piden.

El trabajo fue iterativo e incremental. Se organizó en tres entregas sucesivas y cada una retomó lo hecho en la anterior, incorporó la retroalimentación del docente y profundizó una o más etapas de CRISP-DM. La Tabla 2 muestra esa correspondencia.

Etapa CRISP-DM	Actividad en el proyecto	Entrega
Comprensión del negocio	Análisis del problema de mantenimiento, revisión de literatura, definición de objetivos y requisitos	Sumativa 1
Comprensión de los datos	Carga del dataset AI4I 2020, análisis exploratorio, evaluación de calidad	Sumativa 1–2
Preparación de los datos	Limpieza, eliminación de columnas con fuga de datos, codificación, escalado, partición y balanceo	Sumativa 2
Modelado	Entrenamiento de 3 técnicas de ML y 3 arquitecturas de DL, ajuste de hiperparámetros	Sumativa 2–3
Evaluación	Comparación con métricas sobre datos no vistos, matriz de confusión, curva ROC, interpretación con SHAP	Sumativa 2–3
Despliegue	Servicio API con FastAPI, interfaz web, historial, monitoreo y reentrenamiento	Sumativa 3

Tabla 2. Correspondencia entre las etapas de CRISP-DM y el trabajo realizado.

2.3 Fases del desarrollo

Análisis exploratorio de datos. Se trabajó con el dataset AI4I 2020 (Matzka, 2020). Primero se revisó su estructura: cantidad de registros y columnas, tipo de cada variable y primeros registros. Después se calcularon estadísticas descriptivas (media, mínimo, máximo, desviación) de las variables numéricas, y se generaron histogramas para ver su distribución y boxplots para detectar valores atípicos. La calidad de los datos se evaluó en cuatro aspectos: completitud (valores nulos y duplicados), consistencia (tipos de dato y rangos con sentido físico), exactitud (ausencia de valores imposibles) y balance de clases. Por último se calculó la matriz de correlación de Pearson entre las variables. Esta fase responde al objetivo 1 y a los requisitos de calidad de datos.

Selección de técnicas de aprendizaje automático. A partir de las características del problema (clasificación binaria, datos tabulares, clases muy desbalanceadas y necesidad de explicar las predicciones) y de la revisión de trabajos previos, se definieron las técnicas candidatas. En Machine Learning se eligieron Random Forest, XGBoost y Regresión Logística, que representan tres enfoques distintos: ensamble por bagging, ensamble por boosting y modelo lineal de referencia. En Deep Learning se definieron tres arquitecturas de perceptrón multicapa con distinta profundidad y ancho, para ver si aumentar la complejidad de la red mejoraba el resultado.

Diseño de la arquitectura. Se definieron las entradas y la salida del modelo. Las entradas son siete variables: cinco lecturas de sensores (temperatura de aire, temperatura de proceso, velocidad de rotación, torque y desgaste de herramienta) y el tipo de producto codificado en dos columnas (Type_L y Type_M). La salida es la probabilidad de que la máquina falle, que sobre un umbral de decisión se traduce en una etiqueta binaria. En las redes neuronales se usó siempre la misma función de activación (ReLU) y el mismo optimizador (Adam); lo único que cambió fue el número de capas y de neuronas. Después de comparar los modelos se seleccionó XGBoost y se evaluó un refinamiento con búsqueda de hiperparámetros y validación cruzada.

Implementación. El modelo entrenado se guardó serializado junto con el escalador y el orden de columnas, y se expuso mediante un servicio REST construido con FastAPI. El servicio ofrece endpoints para predicción individual, predicción por lotes, consulta del historial, monitoreo y reentrenamiento. Sobre ese servicio se conectó una interfaz web para el supervisor, con pantallas de predicción, explicabilidad, historial y monitoreo. El preprocesamiento en la inferencia reproduce exactamente el del entrenamiento. Esta fase responde al objetivo 6 y a los requisitos no funcionales de usabilidad, explicabilidad y monitoreabilidad.

Validación. Los datos se dividieron en 70 % entrenamiento y 30 % prueba, con muestreo estratificado para mantener la proporción de fallas en ambos conjuntos. El conjunto de prueba se mantuvo fijo y sin usar durante todo el modelado. El ajuste de hiperparámetros de XGBoost se hizo con validación cruzada de cinco particiones sobre el conjunto de entrenamiento, optimizando F1-Score en lugar de exactitud por el desbalance de clases. Además se compararon tres estrategias de balanceo (sin balanceo, submuestreo y SMOTE) usando el mismo conjunto de prueba y la misma semilla, para aislar su efecto.

Evaluación. Los modelos se compararon con cinco métricas: Accuracy, Precision, Recall, F1-Score y AUC-ROC, calculadas sobre el conjunto de prueba. Se dio prioridad a Recall y F1 de la clase “con falla”, porque

en mantenimiento predictivo el error más costoso es no detectar una falla real. El análisis se complementó con la matriz de confusión y la curva ROC de los seis modelos. Por último se aplicó SHAP (TreeExplainer) sobre el modelo seleccionado para interpretar qué variables influyen en las predicciones, tanto a nivel global como en casos particulares.

2.4 Plan de trabajo

La Tabla 3 resume las tareas realizadas a lo largo del proyecto, con su responsable principal, su plazo y el objetivo o requisito al que responde cada una. La revisión de cada entrega fue conjunta.

Tarea realizada	Responsable	Plazo	Objetivo / requisito
Análisis del problema y revisión de literatura	Javiera Chávez	23 jun – 11 jul 2026	Comprensión del negocio; problemática
Definición de objetivos y requisitos	Equipo completo	7 – 11 jul 2026	Requisitos del proyecto
Carga del dataset y análisis exploratorio	Javiera Chávez	14 – 25 jul 2026	Objetivo 1; calidad de datos
Evaluación de calidad (completitud, consistencia, exactitud, balance)	Javiera Chávez	21 – 25 jul 2026	Objetivo 1
Limpieza, eliminación de fuga de datos, codificación y escalado	Claudio Yáñez	28 jul – 8 ago 2026	Preparación de datos; RF-06
Partición estratificada y comparación de 3 técnicas de balanceo	Claudio Yáñez	4 – 8 ago 2026	Objetivo 3
Entrenamiento y comparación de 3 ML y 3 DL	Claudio Yáñez	11 – 18 ago 2026	Objetivo 2
Ajuste de hiperparámetros con GridSearchCV	Claudio Yáñez	18 – 22 ago 2026	Objetivo 4; validación
Evaluación sobre datos no vistos	Claudio Yáñez	22 – 28 ago 2026	Objetivo 4
Interpretación con SHAP (global y casos particulares)	Javiera Chávez	29 ago – 5 sep 2026	Objetivo 5; explicabilidad
Servicio API con FastAPI	Joaquín Olivares	18 – 28 ago 2026	Objetivo 6; requisitos funcionales
Interfaz web y conexión con la API	Joaquín Olivares	25 ago – 5 sep 2026	Objetivo 6; usabilidad, monitoreabilidad
Pruebas de extremo a extremo del sistema	Equipo completo	5 – 9 sep 2026	Confiabilidad
Repositorio, README y entorno reproducible	Claudio Yáñez	en paralelo	Portabilidad, mantenibilidad
Redacción del informe final y formato APA 7	Javiera Chávez, Claudio Yáñez	1 – 12 sep 2026	Consolidación del proyecto

Tabla 3. Plan de trabajo del proyecto.

3. Desarrollo del proyecto

3.1 Análisis exploratorio de datos

Se trabajó con el AI4I 2020 Predictive Maintenance Dataset (Matzka, 2020), un conjunto de datos público del UCI Machine Learning Repository. Tiene 10.000 registros y 14 columnas, y simula cómo se comporta una fresadora industrial en distintas condiciones de operación.

3.1.1 Descripción de las variables

Variable	Tipo	Unidad	Rango	Descripción
UDI	Entero	–	1 – 10000	Identificador correlativo del registro. Se elimina.
Product ID	Texto	–	–	Identificador del producto (letra de calidad + serie). Se elimina.
Type	Catagórica	–	L, M, H	Calidad del producto: baja (L), media (M), alta (H).
Air temperature [K]	Continua	K	295,3 – 304,5	Temperatura del aire en el entorno del proceso.
Process temperature [K]	Continua	K	305,7 – 313,8	Temperatura del proceso.
Rotational speed [rpm]	Entero	rpm	1168 – 2886	Velocidad de rotación de la herramienta.
Torque [Nm]	Continua	Nm	3,8 – 76,6	Torque aplicado en el husillo.
Tool wear [min]	Entero	min	0 – 253	Minutos de desgaste acumulado de la herramienta.
Machine failure	Binaria	–	0 / 1	Variable objetivo: 1 si la máquina falló.
TWF, HDF, PWF, OSF, RNF	Binarias	–	0 / 1	Tipo específico de falla ocurrida. Se eliminan por fuga de datos.

Tabla 4. Variables del dataset AI4I 2020.

3.1.2 Completitud y correctitud

El dataset viene limpio: no hay valores nulos ni filas repetidas en ninguna de las 14 columnas, así que no hubo que rellenar ni descartar datos. Cada variable tiene el tipo que corresponde (las temperaturas y el torque con decimales, la velocidad y el desgaste como enteros, Type y Product ID como texto) y los valores tienen sentido físico: ninguna medición es negativa y Type solo trae las categorías H, L y M. No aparecieron valores imposibles ni contradicciones.

3.1.3 Estadística descriptiva

Variable	Media	Desv.	Mín	P25	P50	P75	Máx
Air temperature [K]	300,0	2,0	295,3	298,3	300,1	301,5	304,5
Process temperature [K]	310,0	1,48	305,7	308,8	310,1	311,1	313,8
Rotational speed [rpm]	1538,8	179,3	1168	1423	1503	1612	2886
Torque [Nm]	40,0	9,97	3,8	33,2	40,1	46,8	76,6
Tool wear [min]	108,0	63,7	0	53	108	162	253

Tabla 5. Estadísticas descriptivas de las variables numéricas (n = 10.000).

Las variables se mueven en escalas muy distintas: la velocidad de rotación llega a los miles, mientras que el torque y las temperaturas se quedan en las decenas. Por eso conviene escalarlas antes de entrenar los modelos sensibles a la escala. En cuanto a la forma, la temperatura del aire y el torque se parecen a una distribución normal; la velocidad de rotación se concentra en los valores bajos, con una cola que llega hasta 2.886 rpm; y el desgaste de herramienta se reparte de forma casi uniforme entre 0 y 253 minutos.

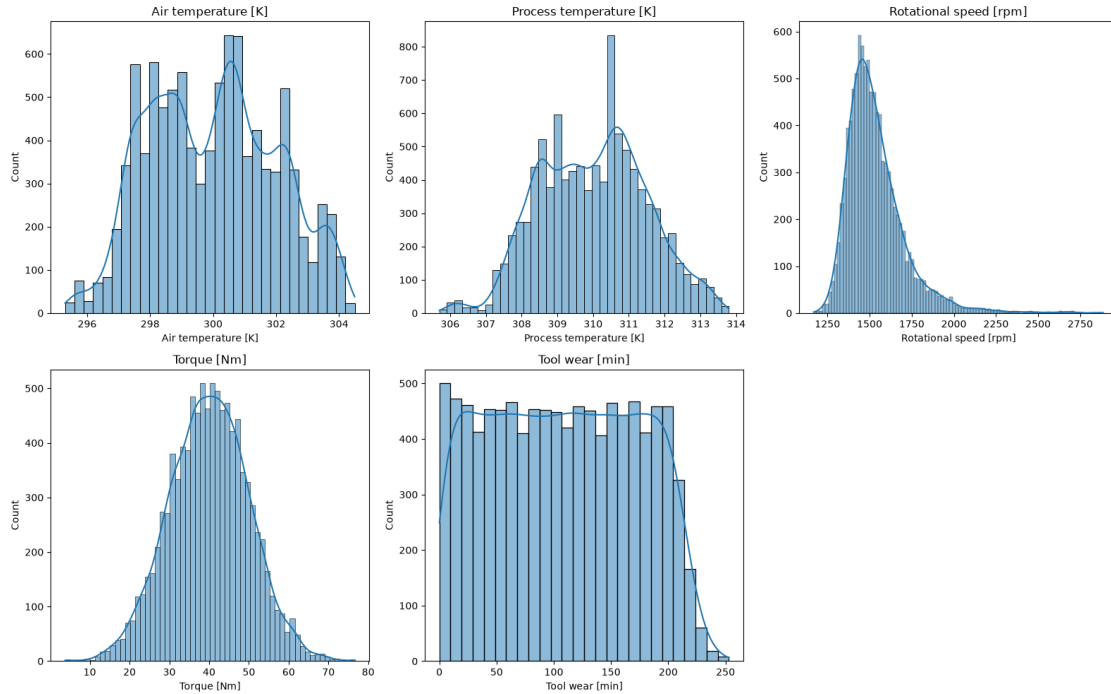


Figura 1. Distribución de las variables numéricas (histogramas con curva de densidad).

3.1.4 Valores atípicos

En los diagramas de caja (Figura 2), la temperatura del aire, la del proceso y el desgaste de herramienta casi no muestran valores atípicos. La velocidad de rotación y el torque sí tienen varios valores extremos. Se decidió conservarlos: en mantenimiento predictivo, esos valores extremos suelen ser justamente las máquinas que están por fallar, y eliminarlos sería perder la señal que más interesa.

Detección de valores atípicos en las variables numéricas

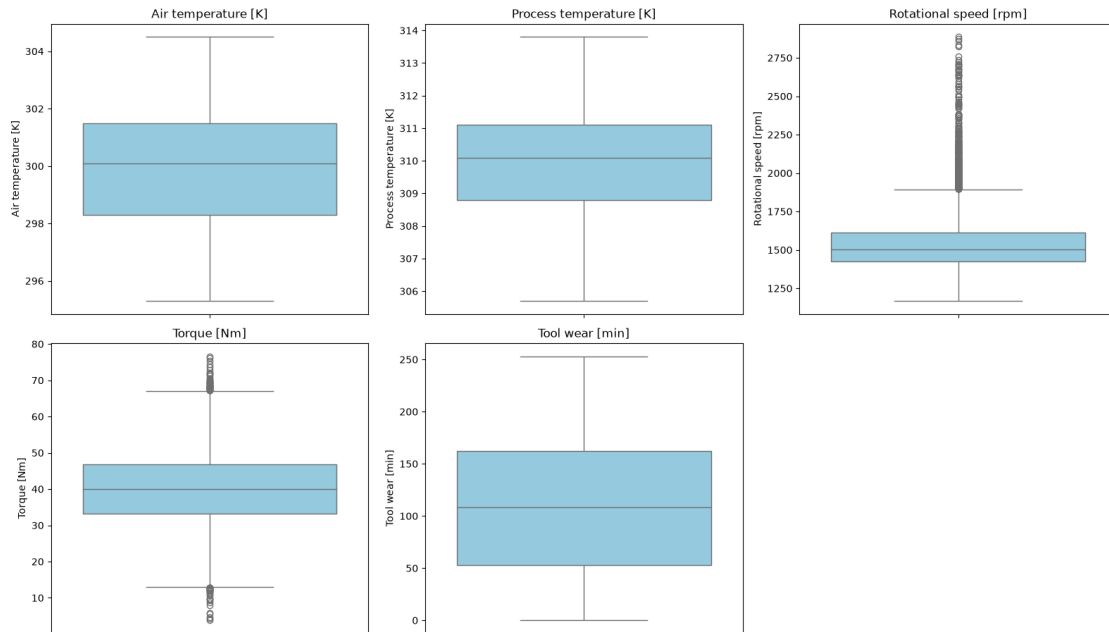


Figura 2. Diagramas de caja de las variables numéricas.

3.1.5 Correlación entre variables

La temperatura del aire y la del proceso suben y bajan casi juntas (correlación de 0,88). La velocidad de rotación y el torque hacen lo contrario: cuando una sube, la otra baja ($-0,88$). Con la variable objetivo, en cambio, las correlaciones son bajas (torque 0,19; desgaste 0,11; temperatura del aire 0,08), lo que confirma que una falla no depende de una sola variable, sino de la combinación de varias. Las columnas TWF, HDF, PWF, OSF y RNF sí muestran correlación alta con Machine failure (HDF 0,58; OSF 0,53; PWF 0,52), pero solo porque describen la falla que ya ocurrió; por eso se eliminan antes de modelar.

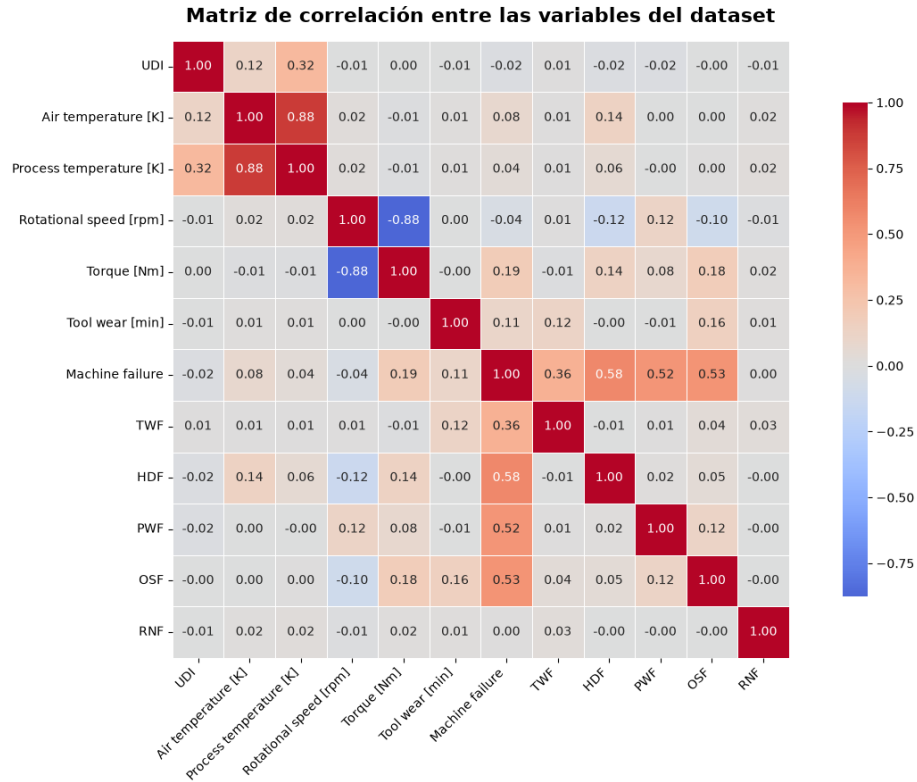


Figura 3. Matriz de correlación de Pearson.

3.1.6 Balance de clases

Hay muy pocas fallas en los datos: de 10.000 registros, 9.661 (96,61 %) son “sin falla” y apenas 339 (3,39 %) son “con falla”. Con este desbalance, un modelo podría predecir siempre “sin falla”, acertar el 96,6 % de las veces y aun así no servir de nada, porque nunca aprendería a reconocer una falla. Por eso, más adelante se comparan tres formas de balancear las clases (sección 3.6) y la evaluación se fija sobre todo en el Recall y el F1 de la clase minoritaria.

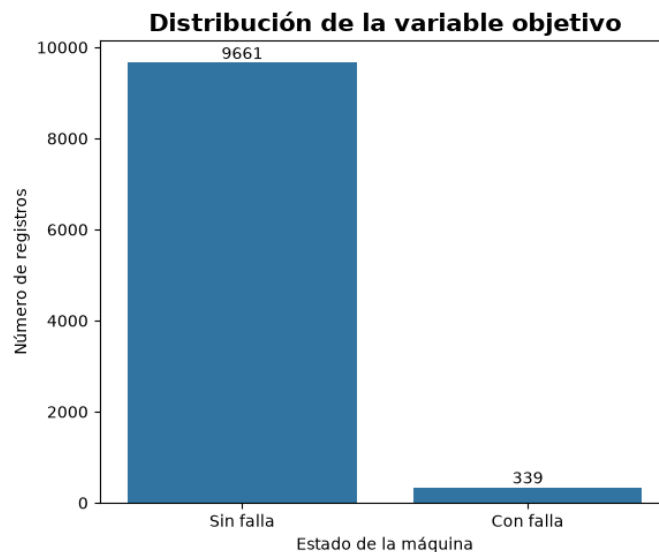


Figura 4. Distribución de la variable objetivo Machine failure.

3.2 Selección de técnicas candidatas

Para elegir las técnicas candidatas se tuvieron en cuenta cuatro características del problema:

- Los datos son tabulares, con 7 variables y 10.000 registros. No es un problema de gran escala, por lo que las técnicas de aprendizaje automático clásico son viables y el aprendizaje profundo no tiene una ventaja evidente por volumen de datos.
- Es una clasificación binaria con clases muy desbalanceadas, así que la técnica debe ser compatible con métodos de balanceo y con métricas sensibles a la clase minoritaria.
- Hay un requisito de explicabilidad, por lo que se prioriza que la técnica admita interpretación (importancia de variables, SHAP).
- El hardware disponible se limita a equipos personales sin GPU, lo que descarta arquitecturas de aprendizaje profundo pesadas (CNN, LSTM, Transformers) en esta etapa.

Los trabajos previos revisados apuntan en la misma dirección. Guidotti et al. (2025), en su revisión sistemática de aprendizaje automático supervisado para mantenimiento predictivo, reportan que los métodos basados en árboles (Random Forest, Gradient Boosting) y las redes neuronales son los más usados y los que mejor rinden con datos tabulares de sensores. Benhanifia et al. (2025) confirman que en la industria manufacturera predominan los ensambles de árboles, por lo robustos que son y lo barato que resulta ajustarlos. Aminzadeh et al. (2025), en su caso aplicado en compresores, muestran que XGBoost y Random Forest le ganan a los modelos lineales y a las redes simples. Matzka (2020), autor del dataset, usa un árbol de decisión y recalca la importancia de la explicabilidad, lo que motivó incluir un modelo lineal interpretable como referencia. Hamasha et al. (2025) suman redes neuronales a su marco, útiles cuando las relaciones entre variables no son lineales.

Con esos antecedentes, las técnicas candidatas de aprendizaje automático fueron:

1. **Random Forest** (Breiman, 2001): ensamble por bagging, robusto al ruido y a los valores atípicos (relevante porque no se eliminaron), entrega importancia de variables.
2. **XGBoost** (Chen & Guestrin, 2016): ensamble por boosting, suele ser el de mejor desempeño en datos tabulares y es compatible con SHAP mediante TreeExplainer.
3. **Regresión Logística**: modelo lineal simple, sirve como línea base y como control. Si un modelo complejo no supera a la Regresión Logística, no se justifica su uso.

Para aprendizaje profundo se definieron tres arquitecturas de perceptrón multicapa (MLP; Goodfellow et al., 2016) con distinta profundidad y ancho, descritas en la sección 3.5.

3.3 Comparación de técnicas

Las seis técnicas se entrenaron en igualdad de condiciones: el mismo conjunto de entrenamiento balanceado con SMOTE (Chawla et al., 2002) y el mismo conjunto de prueba (3.000 registros que ningún modelo vio durante el entrenamiento). La Tabla 6 resume los resultados.

Técnica	Tipo	Accuracy	Precision	Recall	F1	AUC-ROC
Random Forest	ML – Bagging	0,9627	0,4679	0,7157	0,5659	0,9713
XGBoost	ML – Boosting	0,9753	0,6077	0,7745	0,6810	0,9780

Técnica	Tipo	Accuracy	Precision	Recall	F1	AUC-ROC
Regresión Logística	ML – Lineal	0,8353	0,1475	0,8039	0,2492	0,8888
MLP superficial (DL-A)	DL – 1×16	0,9313	0,3169	0,8824	0,4663	0,9692
MLP profunda (DL-B)	DL – 3×64-32-16	0,9660	0,5000	0,7255	0,5920	0,9492
MLP ancha (DL-C)	DL – 1×128	0,9580	0,4388	0,8431	0,5772	0,9744

Tabla 6. Comparación de las 3 técnicas de ML y las 3 arquitecturas de DL sobre el conjunto de prueba.

XGBoost es el que mejor equilibra todo (F1 0,681; AUC-ROC 0,978). Detrás quedan dos redes neuronales: la ancha (AUC-ROC 0,974) y la superficial (0,969), las dos con Recall alto (0,84 y 0,88) pero Precision baja (0,44 y 0,32); es decir, detectan casi todas las fallas, pero a cambio disparan muchas falsas alarmas. Random Forest queda cerca en AUC-ROC (0,971), con menos Precision que XGBoost. La MLP profunda es la peor de las tres redes (AUC-ROC 0,949): sumar capas no ayudó. La Regresión Logística cierra la lista (AUC-ROC 0,889); al ser un modelo lineal, no logra capturar bien cómo se relacionan las variables con la falla.

Restricciones computacionales. Todo se entrenó en computadores personales, sin GPU. Los modelos de ML tardaron segundos y cada MLP menos de un minuto; la búsqueda de hiperparámetros (108 combinaciones × 5 particiones) también quedó por debajo del minuto. No se probaron arquitecturas más pesadas como CNN 1D o LSTM: el problema es tabular y tiene pocas variables, y esas redes necesitan datos en secuencia que este dataset no tiene. Las máquinas de vectores de soporte (Cortes & Vapnik, 1995) quedaron fuera de esta comparación y se dejan como posible trabajo futuro.

Efecto de la calidad de los datos. Con tan pocas fallas, la exactitud engaña: todos los modelos pasan de 0,93 de Accuracy, incluso los que peor detectan las fallas. Por eso la comparación mira sobre todo el Recall y el F1 de la clase “con falla”, además del AUC-ROC.

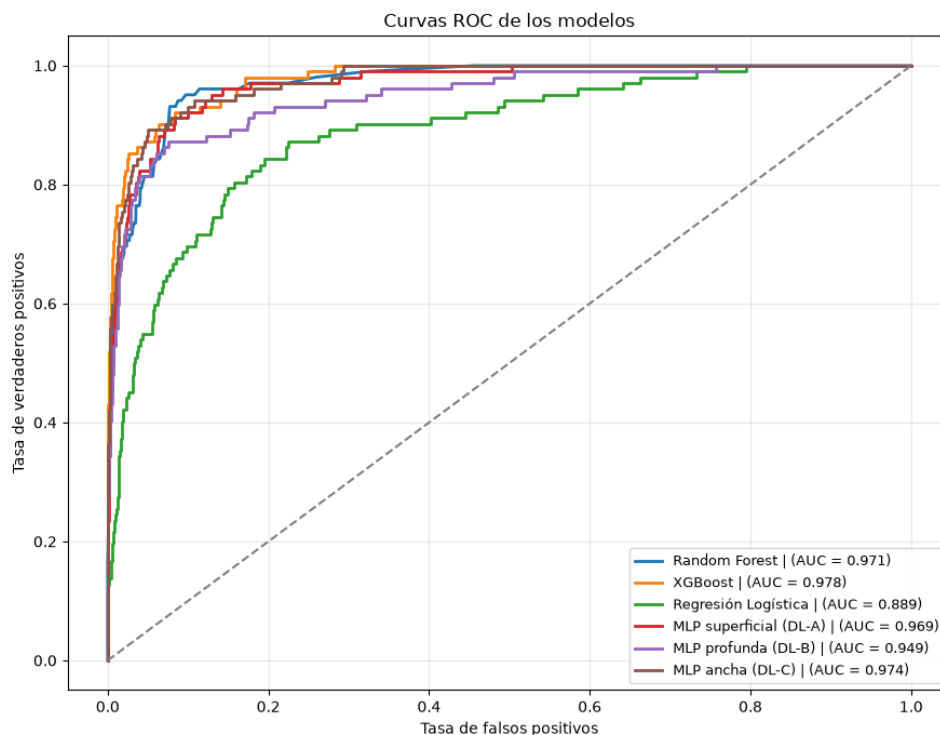


Figura 5. Curvas ROC de las seis técnicas comparadas.

3.4 Requisitos del proyecto

3.4.1 Requisitos funcionales

ID	Requisito	Estado
RF-01	Predicción individual mediante POST /predict	Implementado y conectado al frontend
RF-02	Predicción por lotes (CSV) mediante POST /predict-batch	Implementado y conectado al frontend
RF-03	Validación de estructura y rangos de los datos con esquemas Pydantic	Implementado
RF-04	Preprocesamiento idéntico al de entrenamiento (One-Hot + StandardScaler) en la inferencia	Implementado
RF-05	Estimación de la probabilidad de falla con el modelo XGBoost servido	Implementado
RF-06	Desglose de variables con SHAP en cada predicción	Implementado y conectado al frontend
RF-07	Indicador de nivel de riesgo (No falla / Riesgo medio / Falla) y probabilidad	Implementado
RF-08	Historial de predicciones con filtros por resultado, calidad y origen	Implementado
RF-09	Monitoreo del sistema y del modelo (GET /health, GET /monitoreo)	Implementado
RF-10	Reentrenamiento con datos etiquetados nuevos (POST /reentrenar)	Implementado
RF-11	Exportación del historial (CSV descargable y vista imprimible en PDF)	Implementado

Tabla 7. Requisitos funcionales y su estado.

El requisito RF-06 va de la mano con una decisión que se tomó al limpiar los datos: se quitaron las columnas TWF, HDF, PWF, OSF y RNF porque dicen qué tipo de falla ocurrió, y eso solo se sabe después de que pasó (sería fuga de datos). Por eso el modelo no dice qué tipo de falla va a haber, solo si la máquina va a fallar o no. En su lugar, el desglose SHAP muestra qué variables pesaron más en cada predicción, que es información igual de útil para el supervisor y sin caer en fuga de datos.

3.4.2 Requisitos no funcionales

Categoría	Cómo se aborda y vínculo con el alcance
Usabilidad	Interfaz web con navegación lateral, indicadores de riesgo por color, mensajes de error y plantillas descargables. Cubre el alcance de “interfaz web para el supervisor”.
Explicabilidad	Cada predicción (individual o por lote) incluye el aporte de cada variable calculado con SHAP en el backend. Cubre el alcance de “explicación de cada predicción”.
Escalabilidad	Backend y modelo desacoplados del frontend; el modelo se puede reemplazar sin tocar la interfaz, y el reentrenamiento actualiza el modelo en caliente sin reiniciar la API.
Seguridad	Validación de todas las entradas con Pydantic (rangos físicos por sensor) antes de procesarlas; límite de registros por solicitud; CORS configurable.
Confiabilidad	Verificación de integridad del modelo serializado tras cada reentrenamiento; resultados deterministas a partir de datos validados (semilla fija).
Monitoreabilidad	Endpoints /health y /monitoreo; historial de predicciones y de reentrenamientos persistido en SQLite.
Mantenibilidad	Estructura de carpetas estandarizada (datasets, notebooks, modelos, backend, frontend) y rutas del pipeline independientes del directorio de ejecución.
Portabilidad	Entorno reproducible mediante environment.yml (Conda), documentado en el README.

Tabla 8. Requisitos no funcionales.

3.5 Selección de la arquitectura del modelo

El modelo elegido para producción fue XGBoost. Es el que obtuvo el mejor F1-Score y el mejor AUC-ROC de las seis técnicas, aguanta bien el desbalance de clases y funciona con SHAP a través de TreeExplainer, lo que permite cumplir el requisito de explicabilidad.

Entradas y salida. El modelo recibe siete entradas: temperatura del aire, temperatura del proceso, velocidad de rotación, torque, desgaste de herramienta y el tipo de producto (codificado en Type_L y Type_M). Entrega una sola salida: la probabilidad de falla, un número entre 0 y 1. Con un umbral de 0,5 esa probabilidad pasa a ser una etiqueta (falla / no falla); el frontend, además, usa los cortes 0,34 y 0,67 para mostrar tres niveles: No falla, Riesgo medio y Falla.

3.5.1 Arquitecturas de Deep Learning y análisis de convergencia

Arquitectura	Capas ocultas	Neuronas por capa	Activación	Optimizador	Iteración de parada	AUC-ROC
A – Superficial	1	16	ReLU	Adam	500 (no converge)	0,9692
B – Profunda	3	64, 32, 16	ReLU	Adam	170	0,9492
C – Ancha	1	128	ReLU	Adam	467	0,9744

Tabla 9. Configuración de las tres arquitecturas de DL (MLPClassifier de scikit-learn; Pedregosa et al., 2011).

Las tres redes usan la misma activación (ReLU) y el mismo optimizador (Adam); lo único que cambia es la profundidad y el ancho. La Figura 6 muestra la curva de pérdida (log-loss) de entrenamiento de cada una (notebook, sección 7.4.4). La red profunda (B) baja la pérdida más rápido y se frena por convergencia en la iteración 170, con la pérdida de entrenamiento más baja de las tres (0,03). Pero en el conjunto de prueba es la que peor rinde: está sobreajustando. La red superficial (A) no alcanza a converger en 500 iteraciones (pérdida 0,13) y, aun así, generaliza mejor que la profunda. La red ancha (C) se frena en la iteración 467 con pérdida 0,06 y logra el mejor AUC-ROC de las tres. La conclusión es clara: con solo 7 variables, la red profunda tiene más capacidad de la que necesita y termina memorizando los datos de entrenamiento, mientras que las redes más simples generalizan mejor. Esto respalda la decisión de quedarse con un modelo de árboles.

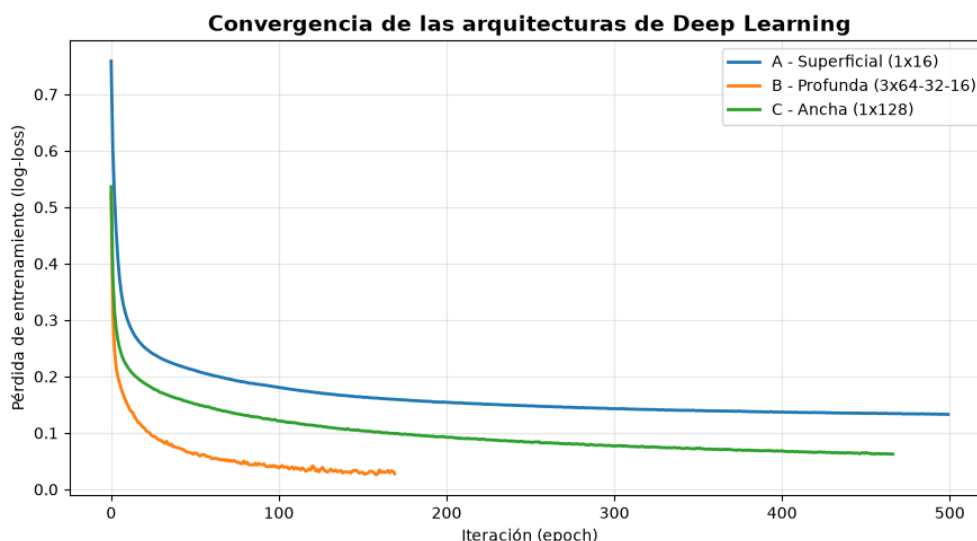


Figura 6. Convergencia (pérdida de entrenamiento) de las tres arquitecturas de DL.

3.5.2 Refinamiento del modelo seleccionado

Después se afinó XGBoost con GridSearchCV: 108 combinaciones de `n_estimators`, `max_depth`, `learning_rate`, `subsample` y `colsample_bytree`, probadas con validación cruzada estratificada de 5 particiones y optimizando el F1-Score. La mejor configuración fue `n_estimators = 300`, `max_depth = 7`, `learning_rate = 0,2`, `subsample = 0,8` y `colsample_bytree = 1,0`, con un F1 de 0,987 en validación cruzada. La Tabla 10 compara el modelo base con el optimizado sobre el conjunto de prueba.

Métrica	XGBoost base	XGBoost optimizado	Diferencia
Accuracy	0,9753	0,9757	+0,0004
Precision (clase Con falla)	0,6077	0,6124	+0,0047
Recall (clase Con falla)	0,7745	0,7745	0,0000
F1-Score (clase Con falla)	0,6810	0,6840	+0,0030
AUC-ROC	0,9780	0,9796	+0,0016

Tabla 10. XGBoost base vs. optimizado con GridSearchCV (conjunto de prueba).

La mejora es pequeña pero pareja: sube algo la Precision, el F1 y el AUC-ROC, y el Recall se mantiene igual. Aun así, se optó por dejar el modelo base como modelo de producción: la ganancia (0,003 en F1) no

justifica sumarle complejidad (300 árboles y profundidad 7 en lugar de la configuración por defecto) al modelo que después hay que servir y explicar. Que el margen sea tan chico dice algo útil de todas formas: el modelo base ya venía bien ajustado para este problema. Por eso el análisis de resultados, la interpretación con SHAP y el modelo serializado en modelos/ usan el XGBoost base. (Los valores salen de volver a ejecutar el notebook completo con la versión actual de las librerías, así que pueden variar un poco respecto de entregas anteriores.)

3.6 Elaboración del modelo: partición y balanceo

El dataset se dividió en 70 % para entrenar (7.000 registros, 237 con falla) y 30 % para probar (3.000 registros, 102 con falla), con muestreo estratificado para que ambos lados mantuvieran el mismo 3,39 % de fallas. El conjunto de prueba se guardó aparte y no se tocó en ninguna parte del modelado ni del balanceo. Para validar durante el ajuste de hiperparámetros se usó validación cruzada estratificada de 5 particiones sobre el conjunto de entrenamiento; no se apartó un tercer conjunto fijo para no achicar todavía más los pocos casos de falla disponibles.

Sobre el conjunto de entrenamiento se compararon tres formas de balancear, dejando fijos el conjunto de prueba, el modelo (XGBoost base) y la semilla aleatoria:

Escenario	N.º entren.	% clase positiva	Accuracy	Precision	Recall	F1	AUC-ROC
Sin balanceo	7.000	3,39 %	0,9833	0,8250	0,6471	0,7253	0,9793
Submuestreo (undersampling)	474	50,00 %	0,9037	0,2533	0,9412	0,3992	0,9655
SMOTE (oversampling)	13.526	50,00 %	0,9753	0,6077	0,7745	0,6810	0,9780

Tabla 11. Efecto de tres técnicas de balanceo sobre XGBoost.

Sin balanceo, el modelo logra la mejor Precision (0,825) y Accuracy (0,983), pero el peor Recall (0,647): se le escapa una de cada tres fallas reales, que es justo lo que no se quiere en mantenimiento predictivo. El submuestreo alcanza el mejor Recall (0,941), pero descarta buena parte de los datos de la clase mayoritaria (de 6.763 a 237 registros por clase), y la Precision se desploma a 0,25, con muchas falsas alarmas (F1 0,40). SMOTE ofrece el mejor equilibrio (F1 0,681) sin perder capacidad de discriminación (AUC-ROC 0,978), así que fue la estrategia elegida. El AUC-ROC casi no cambia entre escenarios porque mide cómo quedan ordenadas las probabilidades, y eso no depende del umbral de decisión.

3.7 Desarrollo frontend y backend

Backend. Es un servicio REST hecho con FastAPI (backend/main.py). Al arrancar carga el modelo XGBoost serializado, el StandardScaler y el orden de columnas que se fijó en el entrenamiento. Los endpoints son: POST /predict (predicción individual), POST /predict-batch (por lotes, hasta 5.000 registros), GET /historial, GET /monitoreo, GET /health, POST /reentrenar y GET /reentrenamientos. Cada entrada se valida con Pydantic contra los rangos físicos de cada sensor, y el preprocesamiento durante la predicción es idéntico al del entrenamiento. El historial de predicciones y de reentrenamientos se guarda en SQLite.

Frontend. La interfaz (frontend/index_1.html) está hecha con HTML, Bootstrap y Chart.js, sin framework. Tiene seis pantallas: Panel principal, Cargar archivo CSV, Predicción individual, Explicabilidad SHAP,

Monitoreo e Historial. Todas piden los datos a la API con fetch. La pantalla de Explicabilidad SHAP muestra el aporte de cada variable que devuelve el backend; la de Monitoreo muestra cuántas predicciones se han hecho, cómo se reparten por nivel de riesgo y por origen, el estado del modelo y el historial de reentrenamientos.

Para mostrar que la integración funciona, esta es una petición directa al backend con los datos de una máquina en riesgo:

```
POST /predict { air_temperature_k: 303.5, process_temperature_k: 312.0, rotational_speed_rpm: 1350, torque_nm: 68.5, tool_wear_min: 220, type: "L" } → 200 OK { prediccion: 1, etiqueta: "Falla", probabilidad_porcentaje: ~100, nivel_riesgo: "Falla", factor_principal: "Torque" }
```

Si se le pasan valores normales de operación, el mismo endpoint responde “No falla” con una probabilidad cercana a 0 %. O sea, el modelo distingue los dos casos y no devuelve siempre lo mismo.

Cómo se cumplen los requisitos. La usabilidad se apoya en la navegación por pantallas y en los colores de riesgo; la explicabilidad, en el desglose SHAP que acompaña a cada predicción (Figura 8); la escalabilidad, en que frontend, backend y modelo están desacoplados y en que el reentrenamiento reemplaza el modelo sin reiniciar la API; la monitoreabilidad, en los endpoints /health y /monitoreo y en el historial persistente (Figura 9). El historial se puede exportar como CSV descargable o como vista imprimible para guardar en PDF.

3.7.1 Capturas del sistema en funcionamiento

Las siguientes capturas muestran el sistema funcionando, con el backend y el frontend conectados y datos reales que vienen de la API.

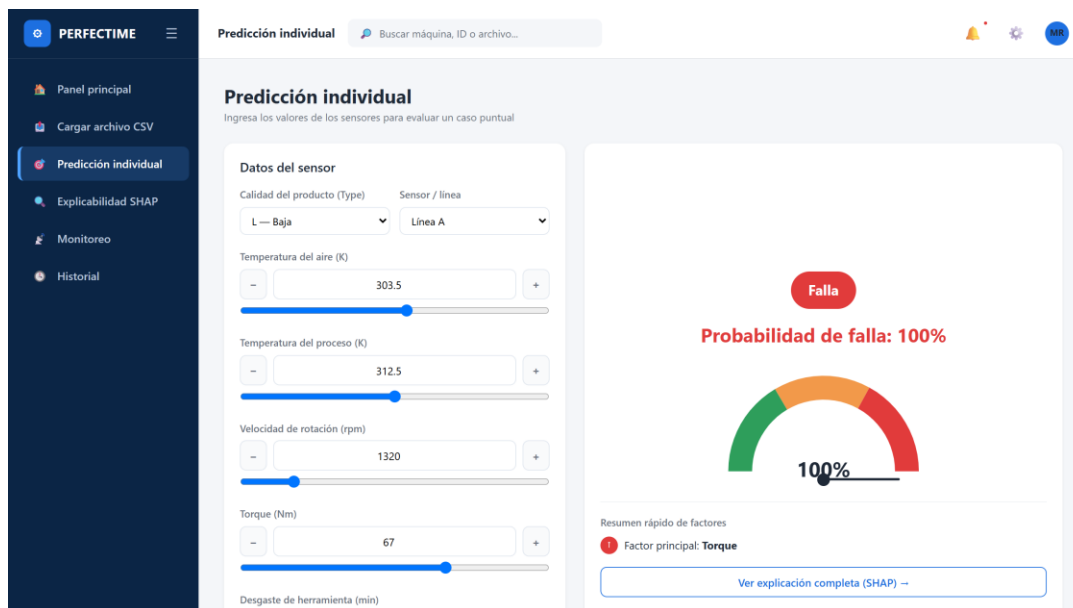


Figura 7. Pantalla de predicción individual: el supervisor ingresa las lecturas de una máquina y obtiene la probabilidad de falla, el nivel de riesgo y el factor principal.

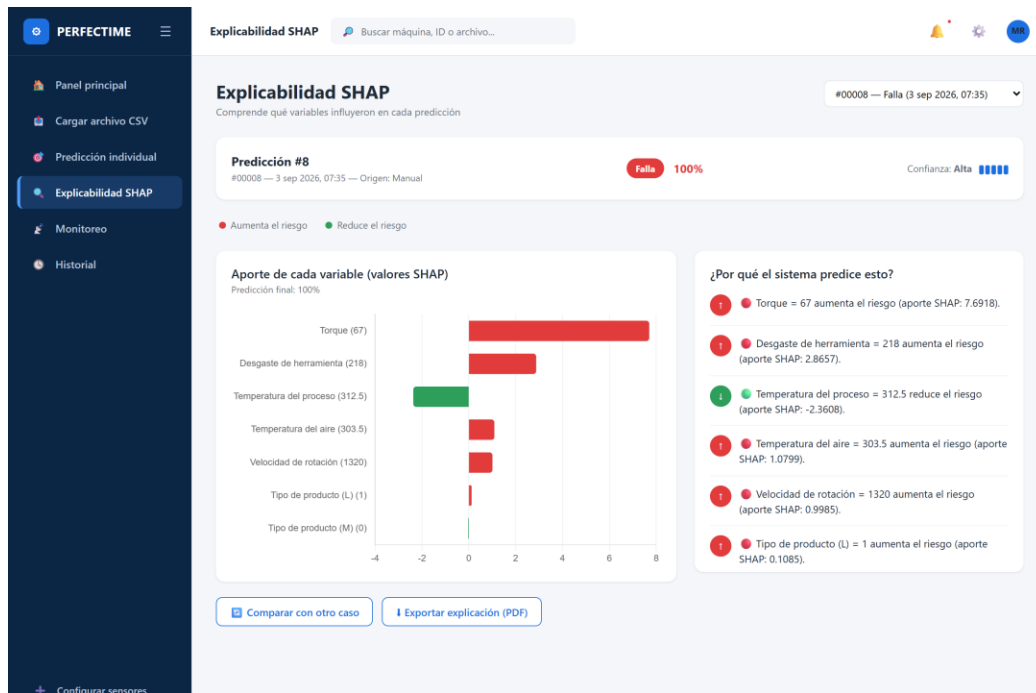


Figura 8. Pantalla de explicabilidad SHAP: aporte de cada variable a la predicción seleccionada, con el detalle en lenguaje natural.

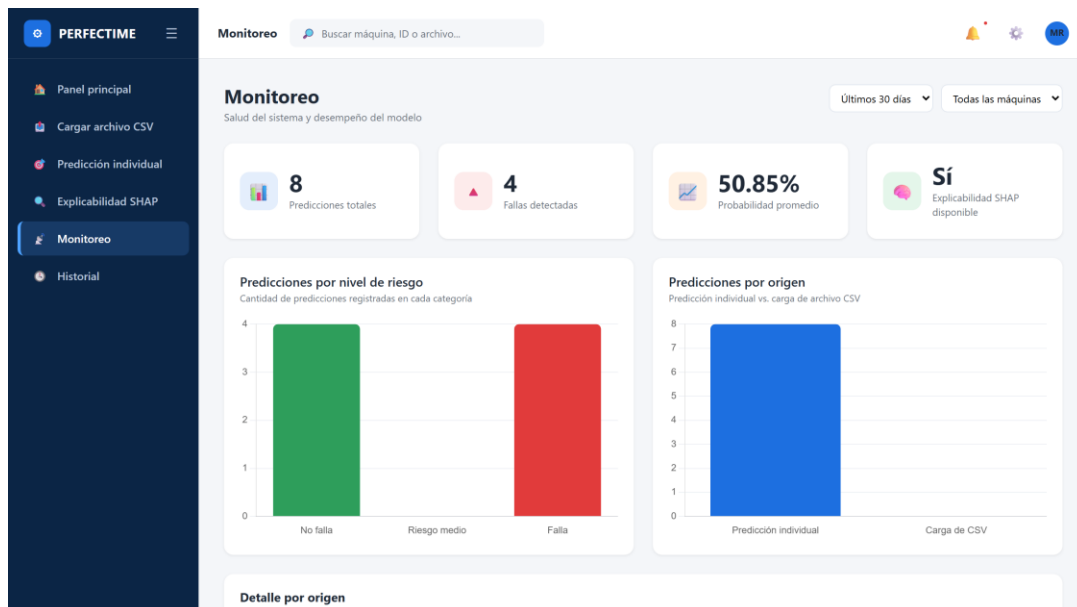


Figura 9. Pantalla de monitoreo: total de predicciones, fallas detectadas, probabilidad promedio, disponibilidad de SHAP y distribución por nivel de riesgo y por origen.

4. Resultados

El modelo de producción (XGBoost base, ver sección 3.5) se puso a prueba con los 3.000 registros del conjunto de prueba, que no se usaron ni para entrenar ni para balancear. La Tabla 12 muestra sus métricas y las compara con la variante optimizada por GridSearchCV.

Métrica	XGBoost base	XGBoost optimizado
Accuracy	0,9753	0,9757
Precision (clase Con falla)	0,6077	0,6124
Recall (clase Con falla)	0,7745	0,7745
F1-Score (clase Con falla)	0,6810	0,6840
AUC-ROC	0,9780	0,9796

Tabla 12. Métricas sobre datos no vistos: XGBoost base (modelo de producción) vs. optimizado.

4.1 Matriz de confusión

La matriz de confusión (Figura 10) muestra que el modelo XGBoost base acierta en 2.847 máquinas sin falla y en 79 con falla. Se equivoca en 51 casos que marca como falla cuando no lo eran (falsos positivos) y en 23 que no alcanza a detectar (falsos negativos). El modelo optimizado baja apenas a 50 falsos positivos. Los 23 falsos negativos son el error que más preocupa en mantenimiento predictivo: son 23 de las 102 fallas reales del conjunto de prueba (Recall 0,77). Un falso positivo cuesta una inspección de más; un falso negativo deja una máquina funcionando sin que nadie la revise antes de que falle.

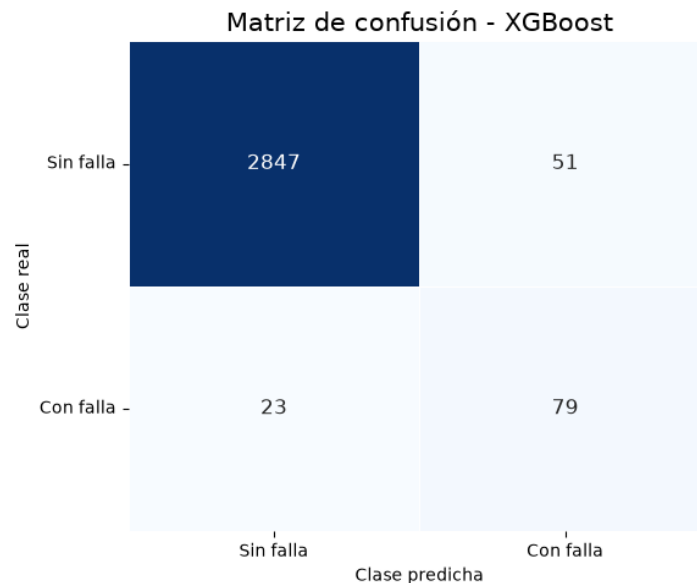


Figura 10. Matriz de confusión del modelo XGBoost (conjunto de prueba).

4.2 Análisis interpretativo con SHAP

Se aplicó SHAP (Lundberg & Lee, 2017) con TreeExplainer sobre el XGBoost base (notebook, secciones 9.2 y 9.3). Mirando todas las predicciones juntas (Figura 11), las variables que más pesan son, en orden: el desgaste de herramienta (2,26), el torque (2,14), la velocidad de rotación (1,37), la temperatura del aire

(1,30) y la del proceso (1,00). El tipo de producto queda al final (Type_L 0,18; Type_M 0,10). Que el desgaste y el torque dominen tiene sentido en un problema de desgaste mecánico; el tipo de producto casi no cambia el resultado.

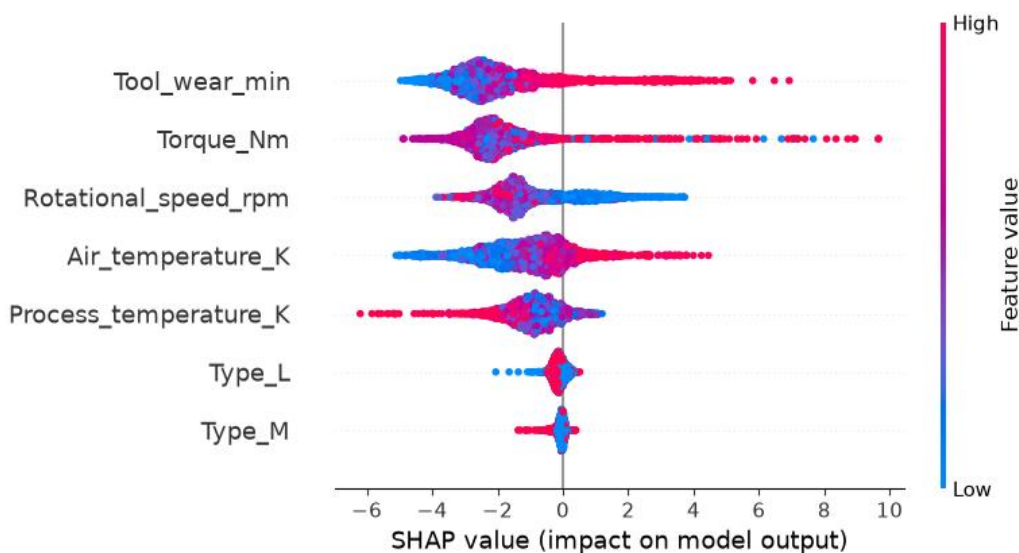


Figura 11. Importancia global de las variables (valores SHAP, modelo XGBoost).

Ver qué variables influyen y en qué dirección es parte de hacer el modelo interpretable (Molnar, 2022). El análisis de casos particulares complementa la visión global:

Verdadero positivo (Figura 12). Máquina con torque de 61 Nm y desgaste de 215 min. El modelo predice falla con probabilidad cercana a 1,00. El torque (+4,0) y el desgaste (+3,3) empujan fuerte la predicción hacia “falla”.

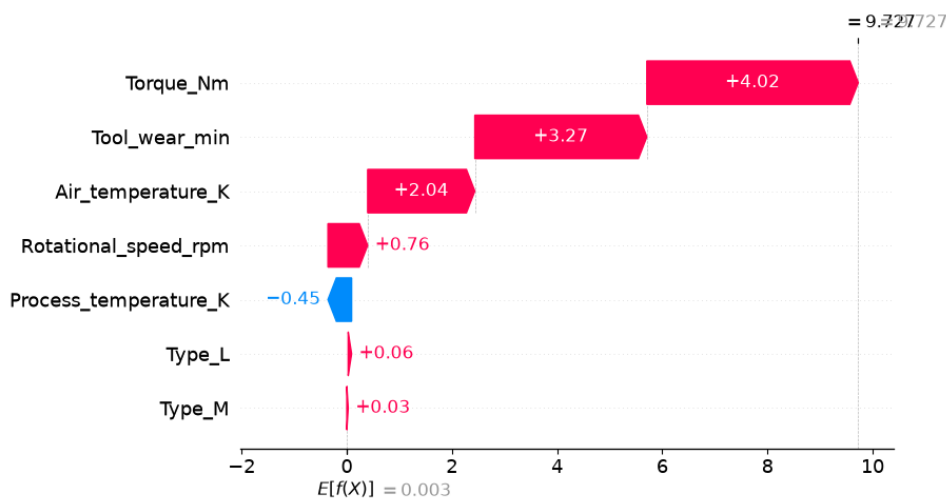


Figura 12. Explicación SHAP de un verdadero positivo.

Falso negativo (Figura 13). Este es el caso más revelador. La máquina falló, pero el modelo predijo “no falla” (probabilidad 0,035). El torque de 41,6 Nm, un valor totalmente normal, empuja -4,4 hacia “no falla”, y el desgaste bajo (34 min) otro -2,4; ni siquiera que la temperatura del aire y la velocidad tiraran

hacia “falla” alcanzó para revertirlo. La falla llegó sin que las lecturas mostraran nada raro, así que el modelo no tenía cómo anticiparla. Es un límite del enfoque, no un error de programación.

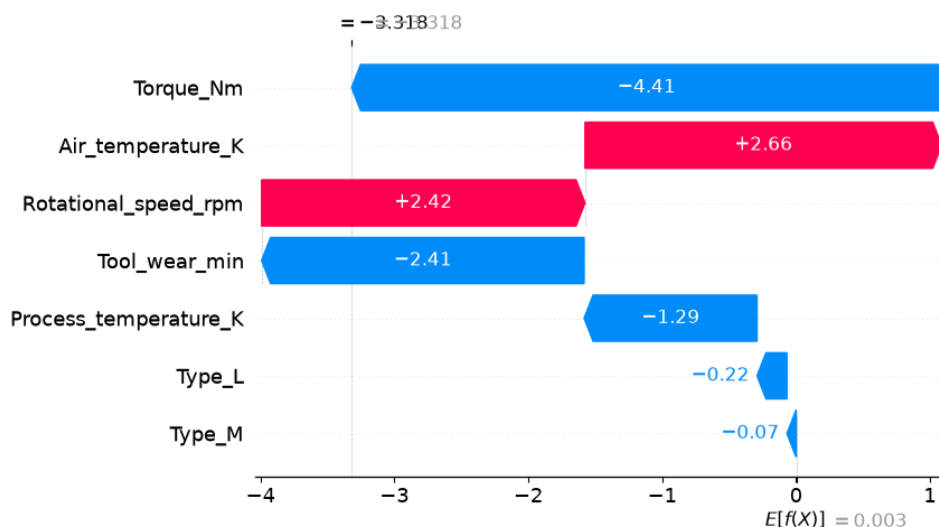


Figura 13. Explicación SHAP de un falso negativo.

Verdadero negativo (Figura 14). Máquina sana, con todas las variables en rango normal; la probabilidad de falla es prácticamente 0. Las siete variables aportan en contra de la falla; el desgaste de herramienta (-4,4) y el torque (-3,0) son los que más pesan.

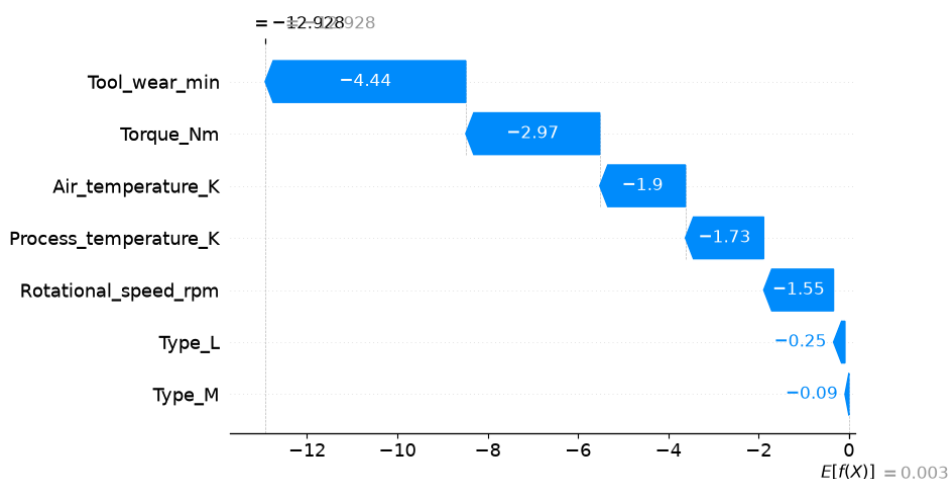


Figura 14. Explicación SHAP de un verdadero negativo.

4.3 Cumplimiento de objetivos y requisitos

El sistema estima la probabilidad de falla con un AUC-ROC de 0,98 y detecta el 77 % de las fallas reales. Eso cumple el objetivo de tener un núcleo predictivo funcionando, aunque el Recall todavía tiene espacio para mejorar. Cada predicción viene con su desglose SHAP, así que también se cumple el requisito de explicabilidad. Y el falso negativo deja clara la principal limitación: con lecturas de un solo instante, hay fallas que no se pueden anticipar. Sobre eso vuelven las secciones de limitaciones y mejoras.

5. Conclusiones, limitaciones y trabajos futuros

5.1 Limitaciones

1. Datos de un solo instante: el dataset entrega mediciones sueltas, sin historia. El modelo no ve cómo evolucionó el desgaste antes de la falla. El falso negativo de la sección 4.2 es el ejemplo: la falla llegó sin que la lectura de ese momento mostrara nada anómalo.
2. Alcance binario: el sistema dice si la máquina va a fallar, pero no de qué tipo será la falla, porque las columnas que lo indicaban (TWF, HDF, PWF, OSF, RNF) se sacaron por fuga de datos.
3. Recall del 77 %: de las 102 fallas del conjunto de prueba, 23 pasan sin detectarse. En una planta real, ese 23 % de fallas no anticipadas seguiría provocando detenciones no planificadas.
4. Dataset simulado: AI4I 2020 es un conjunto generado, no datos reales de una planta. Antes de usar el sistema en producción habría que revalidar todo con mediciones reales.

5.2 Mejoras propuestas

Mejora 1 – Ventanas temporales y arquitectura secuencial (CNN 1D + LSTM). Juntar las últimas N lecturas de cada máquina en una secuencia y entrenar una red que mire la tendencia del desgaste y no solo el valor de un instante. Ataca la limitación 1 y el caso del falso negativo: una falla que no se nota en una lectura suelta sí puede notarse como tendencia. De paso, permitiría estimar la vida útil restante (RUL) del equipo.

Mejora 2 – Ajuste del umbral de decisión y aprendizaje activo. Bajar el umbral de decisión de 0,5 a, por ejemplo, 0,3, para subir el Recall aunque aumenten las falsas alarmas, y mandar las predicciones dudosas (probabilidad entre 0,3 y 0,7) al supervisor para que confirme lo que realmente pasó, alimentando el reentrenamiento. Ataca la limitación 3. El endpoint /reentrenar ya está hecho; falta la parte que elige los casos dudosos y el monitoreo de deriva de datos (data drift).

Mejora 3 (complementaria) – Variables derivadas. Crear variables nuevas a partir de las que ya hay, como la diferencia entre la temperatura de proceso y la del aire, o la potencia mecánica (torque × velocidad). Son habituales en la literatura de mantenimiento predictivo y podrían darle más señal al modelo.

5.3 Resumen de hallazgos y aprendizajes

- De seis modelos (3 de ML y 3 de DL), XGBoost fue el mejor (AUC-ROC 0,98). Las redes neuronales más profundas no le ganaron a las más simples: con pocas variables, más capacidad terminó siendo más sobreajuste.
- La técnica de balanceo hay que elegirla mirando resultados, no solo la teoría: SMOTE dio el mejor F1; el submuestreo llevó el Recall al máximo, pero a cambio de muchas falsas alarmas.
- El ajuste de hiperparámetros aportó poco (F1 0,681 → 0,684): el modelo base ya venía bien ajustado.
- SHAP sirvió no solo para explicar, sino para entender los errores: el falso negativo mostró un límite real del enfoque, no un error de programación.

- Sobre la forma de trabajar: apartar el conjunto de prueba desde el principio y no tocarlo, y comparar los escenarios en igualdad de condiciones (misma semilla, mismo conjunto de prueba), es lo que hace que las conclusiones sean confiables.

5.4 Aplicabilidad del sistema

Perfectime es un sistema de apoyo a la decisión y de gestión del conocimiento para el área de mantenimiento. No reemplaza al supervisor: le entrega, para cada máquina, una probabilidad de falla y las variables que la explican, de modo que pueda priorizar inspecciones con un criterio cuantitativo y trazable.

Ejemplo de uso: al inicio del turno, el supervisor carga un archivo CSV con las lecturas de las 20 máquinas de una línea. El sistema marca tres en “riesgo medio” y, para una de ellas, indica que el factor principal es el desgaste de herramienta. El supervisor programa el cambio de herramienta de esa máquina antes de que falle. El conocimiento que antes dependía de la experiencia individual queda registrado en el historial y sirve, además, para reentrenar el modelo con datos de la propia planta.

6. Código fuente en GitHub

El código fuente completo del proyecto (notebook de análisis y entrenamiento, backend, frontend, dataset, modelos serializados y archivos de configuración) está disponible en el siguiente repositorio:

https://github.com/XetuRiot/Acif104_Grupo5

El repositorio está organizado en carpetas estandarizadas: datasets/ (dataset AI4I 2020), notebooks/ (Sumativa.ipynb con el pipeline completo y sus resultados), modelos/ (artefactos serializados que genera el notebook: modelo_xgboost.pkl, scaler.pkl y columnas.pkl), backend/ (servicio FastAPI), frontend/ (interfaz web) y Documentacion/ (informes). El archivo README.md contiene las instrucciones para crear el entorno Conda desde environment.yml, ejecutar el notebook para entrenar el modelo, y levantar el backend y el frontend; existe además README_VSCode.md con el mismo flujo desde VS Code.

El notebook se ejecutó completo con la versión actual de las librerías, así que todas las tablas y figuras de este informe salen de esa corrida. Antes de entregar, falta verificar que el repositorio esté público y sumar el enlace al video corto que muestra el flujo de predicción.

7. Bibliografía

- Aminzadeh, A., Sattarpanah Karganroudi, S., Majidi, S., Dabompre, C., Azaiez, K., Mitride, C., & Sénéchal, E. (2025). A machine learning implementation to predictive maintenance and monitoring of industrial compressors. *Sensors*, 25(4), 1006. <https://doi.org/10.3390/s25041006>
- Benhanifia, A., Ben Cheikh, Z., Oliveira, P. M., Valente, A., & Lima, J. (2025). Systematic review of predictive maintenance practices in the manufacturing sector. *Intelligent Systems with Applications*, 26, 200501. <https://doi.org/10.1016/j.iswa.2025.200501>
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>
- Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS Inc.
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357. <https://doi.org/10.1613/jair.953>
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. En *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). ACM. <https://doi.org/10.1145/2939672.2939785>
- Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297. <https://doi.org/10.1007/BF00994018>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
- Guidotti, D., Pandolfo, L., & Pulina, L. (2025). A systematic literature review of supervised machine learning techniques for predictive maintenance in Industry 4.0. *IEEE Access*, 13, 102479–102504. <https://doi.org/10.1109/ACCESS.2025.3578686>
- Hamasha, M. M., Albedoor, Q., Hamasha, S., Ali, H., Qamar, A., & Berrah, F. (2025). A comprehensive framework for IoT-driven predictive maintenance: Leveraging AI and edge computing for enhanced equipment reliability. *Journal of Applied Engineering Science*, 23(3), 471–486. <https://doi.org/10.5937/jaes0-57002>
- He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263–1284. <https://doi.org/10.1109/TKDE.2008.239>
- Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. En I. Guyon, U. von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, & R. Garnett (Eds.), *Advances in Neural Information Processing Systems 30* (pp. 4765–4774). Curran Associates.
- Matzka, S. (2020). Explainable artificial intelligence for predictive maintenance applications. En *2020 Third International Conference on Artificial Intelligence for Industries (AI4I)* (pp. 69–74). IEEE. <https://doi.org/10.1109/AI4I49448.2020.00023>

- Meitz, L., Senge, J., Wagenhals, T., Schöler, T., Hähner, J., Edinger, J., & Krupitzer, C. (2025). A literature review framework and open research challenges for predictive maintenance in Industry 4.0. *Computers & Industrial Engineering*, 206, 111193. <https://doi.org/10.1016/j.cie.2025.111193>
- Molnar, C. (2022). *Interpretable machine learning: A guide for making black box models explainable* (2.^a ed.). <https://christophm.github.io/interpretable-ml-book/>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.